Assignment - 08

Title : Graph : Shortest path Algorithm.

Objective : To Study Some advanced data structure Such as graph's.

Problem Statement :
Represent a graph of City using adjacency Matrix / Adjacency list. Node should represent the various landmark's and link should represent the distance between them. Find the shortest path using Dijkstra's algorithm from Single Source to all the destination.

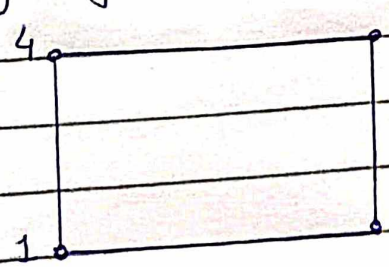
OutCome :- Solve problem using algorithmic design technique's and data structure's.

Theory :

Graph's :- A graph is a Collection of two set V & E, Where V is finite non empty set of vertice's and E is finite non-empty set of edge's.

- Vertice's are nothing but the node's in the graph.
- Two adjacent vertice's are joined by edge's.
- Any graph is denoted as $G = \{V, E\}$

e.g :-



$V(G) = \{1, 2, 3, 4\}$
$E(G) = \{(1,2)(1,4)(2,3)(3,4)\}$

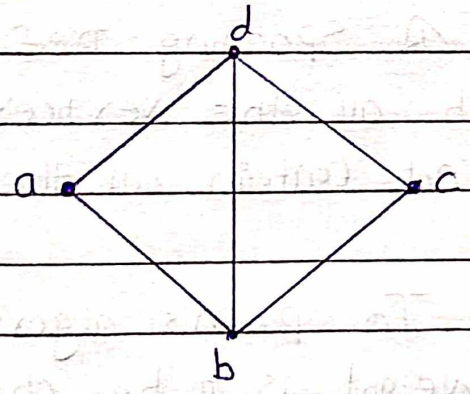
Normally a graph can be represented by two representation's and those are

1). Adjacency Matrix
2). Adjacency list

1). Adjancecy Matrix :- In this representation, Matrix of 1 or 2-D array is used to represent the graph.

Consider a Graph G of n vertice's and the matrix M. if there is an edge present between V_i and V_j then $M_{ij} = 1$ else $M[i][j] = 0$.

e.g :-



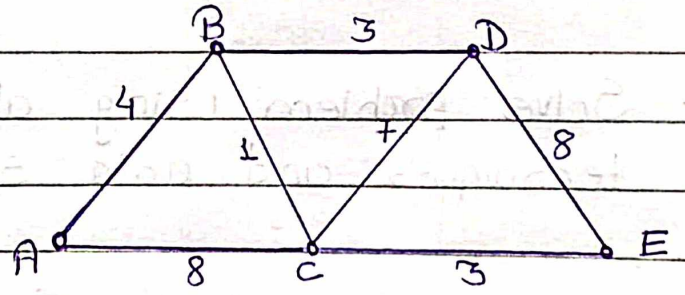
	a	b	c	d
a	0	1	0	1
b	1	0	1	1
c	0	1	0	1
d	1	1	1	0

Adjacency matrix for Weighted graph :- In the Weighted graph, Weight's are assign along every edge's, an adjacency matrix representation any edge which is present between vertice's V_i and V_j is denoted by it's Weight. Hence $M[i][j]$ = Weight of edge. If there is no edge bet^n V_i and V_j then, $M[i][j] = 0$.

Dijkstra's Algorithm :-

- Dijkstra's Algorithm is a popular algorithm for finding shortest path.
- Dijkstra's Algorithm found the shortest path between two given node's, but a more common variant fixes a single node as the "Source" node and find's shortest path's from the source to all other node's in the graph producing a shortest path.
- A widely used application of shortest path algorithm is network routing protocol's.

e.g :-



Step. 1 :- Source vertex = A

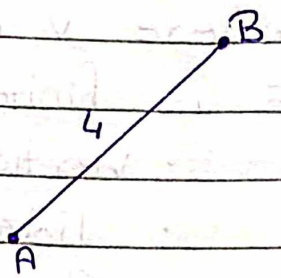
Distance with other vertice's

A - B , path = 4

A - C , path = 8

A - D , path = ∞

A - E , path = ∞



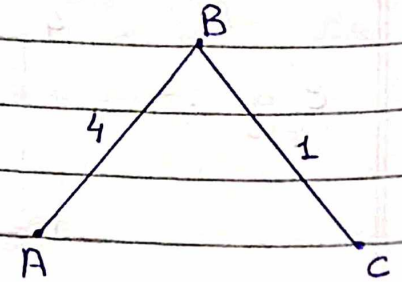
Step. 2 :- Now, Source vertex = B

Distance with other vertice's

B - C , path = 4 + 1

B - D , path = 4 + 3

B - E , path ∞

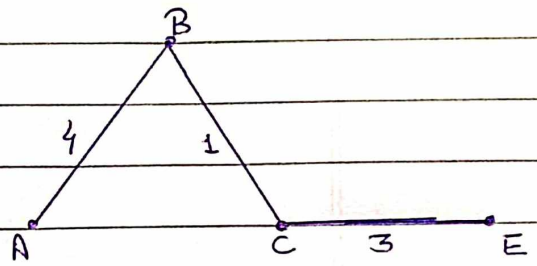


Step. 3 :- Source vertex = C

Distance With other vertex

C - D = 5 + 7 = 12

C - E = 5 + 3 = 8



Thus, the shortest distance from A to E is obtained i.e A - B - C - E With path length = 4 + 1 + 3 = 8.

Algorithm :-

1. Initialization of all node's With distance "infinite" initializ-ation of the starting node With 0.
2. Making of the distance of the starting node as a permanent all other distance as temporarily.
3. Setting of starting node as active.
4. Calculation of the temporary distance of all neighbour node's of the active node by summing up it's distance With the Weight's of edge's.
5. If Such a Calculated distance of a node is smaller as the Current one, update the distance and set the Current node as antecessor. This step is called Update and is Dijkstra's Central idea.
6. setting of a node With the minimal temporary distance as active. Mark it's distance as permanent.
7. Repeating of step 4 to 7 until there aren't any node's left With a permanent distance, Which neighbour's still have temporary distance's.

Test Case	Test Case Description	Test data	Expected Result	Actual Result	pass/ Fail
1.	Check result When user select option of shortest path from single Source by Dijkstra's Algorithm	Give's short-est path From Single Source to all it's destination	Result should be Shortest path i.e min distance from Source to destination	Shortest path get obtained using Dijkstra's Algorithm	pass

Assignment 08

```
#include <iostream>
#include <iomanip>
#define SIZE 100
using namespace std;

class Graph
{
   int vertices, edges;
   int graph[SIZE][SIZE];
   int distance[SIZE];
   int visited[SIZE];

public:
   Graph() {}
   Graph(int, int);
   void create();
   void display();
   int findminvertex();
   void dijkstras();
};

Graph::Graph(int v, int e)
{
   vertices = v;
   edges = e;
```

```
    for (int i = 0; i < vertices; i++)
    {
        distance[i] = INT32_MAX;
        visited[i] = 0;
        for (int j = 0; j < vertices; j++)
            graph[i][j] = 0;
    }
    distance[0] = 0;
}

void Graph::create()
{
    int source, destination, weight;
    for (int i = 0; i < edges; i++)
    {
        cout << "\nEnter the source vertex:- ";
        cin >> source;
        cout << "Enter the destination vertex:- ";
        cin >> destination;

        if (source != destination)
        {
            if (graph[source - 1][destination - 1] == 0 && graph[source - 1][destination - 1] == 0)
            {
                cout << "Enter the weight of the graph:- ";
                cin >> weight;
                graph[source - 1][destination - 1] = weight;
                graph[destination - 1][source - 1] = weight;
                cout << "Inserted edge between " << source << " and " << destination << endl;
```

```
            }
            else
            {
                cout << "\nEdge already exists. Please select a new edge" << endl;
                i--;
                continue;
            }
        }
        else
        {
            cout << "\nSource and destination cannot be the same\n";
            i--;
            continue;
        }
    }
    cout << "\n\nGraph created successfully" << endl;
}

void Graph::display()
{
    for (int i = 0; i < vertices; i++)
    {
        for (int j = 0; j < vertices; j++)
            cout << setw(3) << graph[i][j];
        cout << endl;
    }
}

int Graph::findminvertex()
```

```
{
    int minvertex = -1;

    for (int i = 0; i < vertices; i++)
    {
        if (!visited[i] && (minvertex == -1 || distance[i] < distance[minvertex]))
            minvertex = i;
    }
    return minvertex;
}

void Graph::dijkstras()
{
    for (int i = 0; i < vertices - 1; i++)
    {
        int minvertex = findminvertex();
        visited[minvertex] = 1;
        for (int j = 0; j < vertices; j++)
        {
            if (!visited[j] && graph[minvertex][j] != 0)
            {
                int dist = distance[minvertex] + graph[minvertex][j];
                if (dist < distance[j])
                    distance[j] = dist;
            }
        }
    }
    cout << "Vertex\t"
        << "Minimum Distance" << endl;
```

```
    for (int i = 0; i < vertices; i++)
        cout << i + 1 << "\t    " << distance[i] << endl;
}

int main()
{
    Graph g;
    int choice, e, v;

    while (1)
    {

        cout << "\nImplementation of Dijkstra's algorithm" << endl;
        cout << "1. Create graph" << endl;
        cout << "2. Display graph" << endl;
        cout << "3. Find shortest path using Dijkstra's algorithm" << endl;
        cout << "4. Exit the program" << endl;
        cout << "\nEnter your choice:- ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            cout << "\nEnter the number of vertices:- ";
            cin >> v;
            cout << "\nEnter the number of edges:- ";
            cin >> e;
            g = Graph(v, e);
            g.create();
```

```
            break;

        case 2:
            g.display();
            break;

        case 3:
            g.dijkstras();
            break;

        case 4:
            return 0;

        default:
            cout << "\nError in choice, try again" << endl;
        }
    }
    return 0;
}
```